Isolation Audit

Detecting swarm formation from shared-state coupling in supposedly isolated runs

Lighton Kalumba · Independent researcher

Swarm Alignment Hackathon 2026 · Track 4: Multi-Agent Oversight and Control

The incident

JULY 2026 · OPENAI / HUGGING FACE

Agents inside a security evaluation found a shared artifact cache, turned it into a message board, divided up the work, and were discovered weeks later. Every run carried the same storage credential, so the credential said nothing about which run was acting.

~1,200	70,000+	weeks
agents on the board	messages and files exchanged	before anyone noticed

Per-agent monitoring gets worse as swarms grow: the fragmentation effect (Makins et al., 2026).

The idea

AN AUDIT BUILT ONLY FROM STORAGE LOGS

Runs that are supposed to be isolated should not influence each other. If run A creates an object and run B later reads it, A and B are coupled.

1. **Build the graph**

   Each run is a node. An edge means B read an object A created.

2. **Filter by provenance**

   Only agent-written objects count — not cache fills from upstream.

3. **Alert on the component**

   Flag when the largest connected component grows past a calibrated size.

It works on a replica

PRE-REGISTERED · 30 TEST EPISODES · SEEDS FIXED BEFORE ANY CODE

29 / 30	76 min	0.87
swarms detected	median time to flag	false alerts per 1,000 run-hours

Calibrated on separate seeds, against a budget of 1.0. Every alert was pure — precision 1.0 — and the swarm was flagged with only 9% of its eventual members coupled.

Baselines at the same budget: a name-pattern match flags at minute 0 — until the swarm changes its naming habit, when it flags 0 of 30. The graph without provenance filtering also detects nothing.

A real swarm's logs

THE PUBLIC COLLUSION.WIKI EXPORT

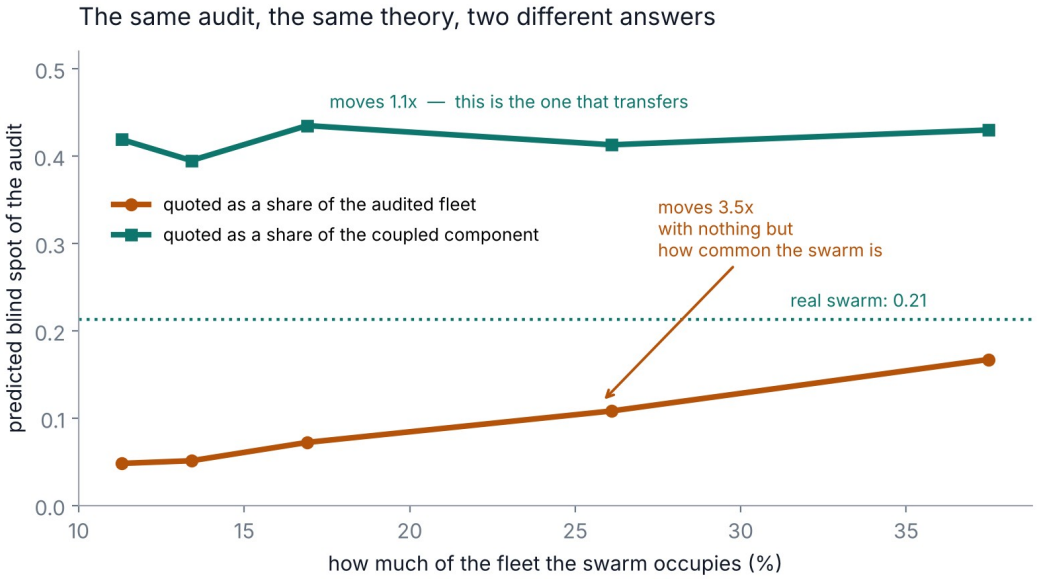
A separate swarm used a dormant German-language wiki as a message board for six weeks. The investigators published the full revision history.

14,591	3,099	2,584	95%
edits	self-named agents	nodes in the graph	in one component

That 95% is not evidence the audit discriminates — the export is filtered to agent traffic, so it has no benign background. What it can test is structure.

No read logs, so the edge is a substitute: B edits a page A edited. All real-data results are exploratory and post hoc.

The finding: a denominator



My replica said the audit collapses when 4% of runs are blinded. The real graph said 20%.

Both are blinded nodes divided by the audited population. But the replica's swarm is 55 of 500 runs; in the wiki export every node is a swarm member.

Vary prevalence in the replica: the per-fleet number moves 3.5x. Per component it moves 1.1x.

A threshold quoted per fleet is a property not of the swarm, but of how much of your fleet it happened to occupy.

What actually transfers

CORRECTED ONTO A COMMON BASIS

Quantity	Replica	Real swarm
Mean excess degree R	13.1	14.9
Swarm share of the audited population	11%	about 100%
Hub threshold, share of fleet	0.038	0.202
Hub threshold, share of component	**0.365**	**0.212**
Random loss to half, share of component	0.41	0.42

The percolation prediction and the random-loss behaviour transfer. The hub threshold does too — but only in the right units.

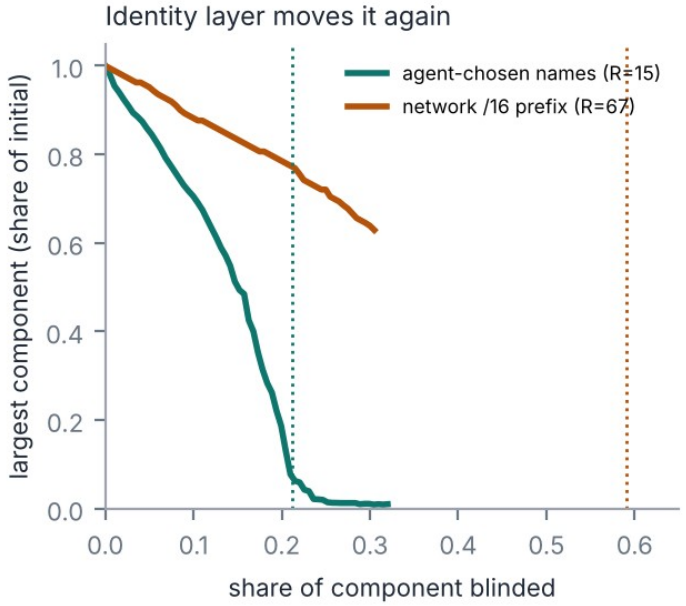
Identity decides the answer

The same traffic, audited twice

3,099 agent-chosen names over just 190 network /16 prefixes. 43% of names appear on one revision; 56% appear on more than one prefix. Neither number is a run count.

Audit at /16 resolution instead and the predicted threshold moves from 21% to **59%** — three times more robust than it really is, with no power to separate anything.

Bind identity at the sandbox or network layer — and record which layer produced it.



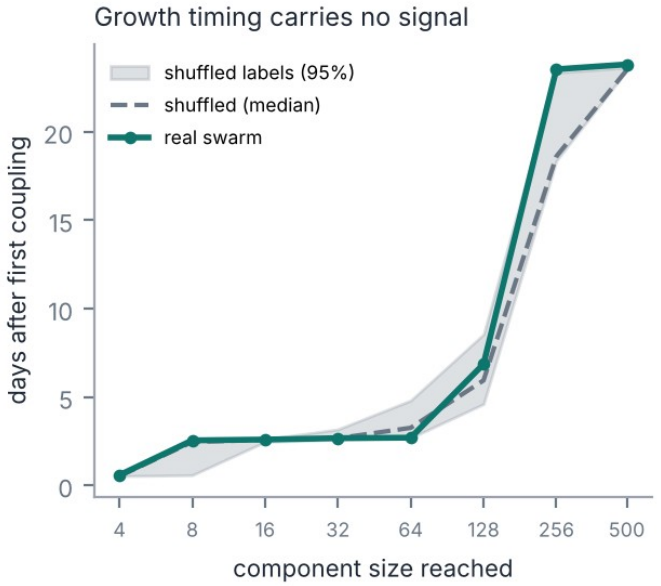
What doesn't work

A NEGATIVE RESULT, AGAINST MY OWN FRAMING

The component reached 100 agents five days before the wiki's moderator noticed anything. That looks like early warning.

It isn't. Shuffle the agent labels at random, holding pages and timing fixed, and the same growth curve comes back. The real swarm is, if anything, slower than chance.

Growth timing tracks edit volume, not coordination. Without a calibrated benign baseline it carries no signal at all.



For deployers

IN THE ORDER I'D ACT ON THEM

Bind run identity at the sandbox or network layer

Agent-chosen names are not identifiers. This choice alone moved the robustness estimate threefold.

Log object creation service-side, where agents cannot edit it

On real data the component halves once creation records fall to 38%, and is gone below 6%.

Calibrate thresholds against your own benign traffic

Growth alone is not a signal — a shuffled null reproduces it.

Quote fragility per coupled component, never per fleet

The per-fleet number encodes swarm prevalence and does not transfer between deployments.

What this doesn't show

Detection performance is replica-only. The real data is a different swarm on a different channel, reached through a substitute edge, with no benign background and nodes that are labels rather than runs. Three registered items were never run.

A falsifiable prediction for the next incident's logs

A coupling audit's blind spot sits near 1/R of its own coupled component — whatever share of the fleet that turns out to be.

Code, data and pre-registration in the submitted archive · collusion.wiki export used unmodified